

从 Skill 1.0 到 Skill 2.0：模块化 Skill 中间件与投研场景应用

——以固定收益研究为例

From Skill 1.0 to Skill 2.0: A Modular Skill Middleware for Research Applications

叶青

2026年8月

摘要

：第一代 Skill 以"一个目录装下一整套能力"的形态交付：全局可用、整体加载，封装与复用都以整个 Skill 为单位，内部结构与依赖关系对外不透明。当主题数据库扩展到十余个、分析视角累积到数百个之后，整体加载成本上升、复用颗粒度过粗的问题开始制约生产运行。本文提出 Skill 2.0，把 Skill 从"整体可用的黑盒"重构为"模块化中间件"：将能力拆解为 SKILL.md、knowledge、templates、data_contract、scripts、config 六类模块，每类模块具备独立的 IN/OUT/DEP 接口契约，可被单独调用、单独测试、单独版本化；中间件在数据源、取数、知识、计算、输出五个层面独立可替换，通过 data_contract 隔离数据源绑定，并通过知识层回验基线保证"框架优先于数值"。基于固定收益研究线上体系（13 个主题数据库、约 500 个分析逻辑、多本生产运行的技术架构手册）的实践表明，模块化设计显著降低了集成成本、提升了复用粒度，并具备向权益、宏观、量化等领域迁移的潜力。

关键词：AI Agent、Skill 模块化、中间件、投研数字化、固定收益

1. 引言

1.1 问题

Skill 1.0 解决了"研究能力能否封装"的问题——拉数、算指标、比阈值、写判断、出报告这条链条，逐一 Skill 化并在生产环境跑通。然而随着运行规模扩大，第一代 Skill 在工程特性上的局限逐渐显现，集中表现为四个"过"字：

- **适用范围过宽：**Skill 默认全局可用，任何任务都可能涉及它。激活一次，就把与当前任务无关的内容一并引入，形成持续的上下文噪声。
- **加载粒度过粗：**运行单元是整个 Skill 目录。哪怕只需要其中一个视角的公式，也要加载全套知识、模板与脚本。
- **内部结构过黑：**对外只见 SKILL.md 的总体描述，模块内部的依赖关系、数据来源、计算口径不可见，集成者只能整体借用，无法局部审查。

- **复用单位过大**：想复用一个视角（例如"隔夜资金价格的分位判断"），在 Skill 1.0 下往往整体复制整个数据库 Skill，跨库引用造成大量重复与版本漂移。

这四个局限的本质是同一件事：**封装粒度与运行单元都是"整个 Skill"**，与**"一次任务通常只需要其中一小块能力"**的现实不匹配。

1.2 思路

思路是拆解：研究能力的最小可复用单位，不应是"整个 Skill"，而应是"一个模块"。Skill 2.0 是一个模块化中间件——对内，把 Skill 拆成职责单一、契约独立的模块；对外，按需组装，只取一个模块即可独立工作。

"中间件"在此是两层含义的压缩表达。其一，研究链条（数据源→取数→知识→计算→输出）的每一层都可以独立存在、独立替换，任一层的实现变更不迫使其它层联动修改；其二，能力以中间态（可供组装的结构化产物）沉淀，研究想法是这些基础模块的组合。这一立场与 v1 论文中"研究能力逐层拆解、需要时再组装"的主张一脉相承，不同之处在于：Skill 2.0 把拆解的粒度推进到了**模块**这一层，并为每个模块规定了可机器校验的接口契约。

2. 从 Skill 1.0 到 Skill 2.0

2.1 Skill 1.0 的形态与局限

Skill 1.0 的典型形态是一个扁平目录：一份 SKILL.md 加上若干散放的文档与脚本，通过"全局注册、语义匹配激活"进入 AI 工作流。它在生产运行中的局限如下表：

维度	Skill 1.0 的形态	生产运行中的局限
可用范围	全局注册，随任务语义自动激活	无关任务也被涉及，激活噪声上升
加载粒度	整个 Skill 目录整体加载	只用一角也要装下全套，上下文开销高
结构契约	无统一模块规范，脚本与文档混排	内部依赖不可见，集成只能整体借用
复用单位	整个 Skill	颗粒度过粗，跨库复用靠拷贝与剪裁
数据绑定	数据源在脚本内固化	切换数据商需改动取数代码

这五个局限并非封装思路本身的失败，而是封装粒度选择的结果。Skill 1.0 回答的是"能不能封装"，Skill 2.0 追问的是"封装到什么粒度、以什么契约封装才更适合生产"。

2.2 Skill 2.0 的设计：六类模块

Skill 2.0 规定一个 Skill 的标准结构由六类模块组成：

```
skill-XXX/
├── SKILL.md           ← 激活与路由（触发条件 + 视角清单 + 模块索引）
```

└─ knowledge/	← 知识模块（公式 + 阈值 + 案例 + 回验基线）
└─ templates/	← 输出模板模块（表 / 图 / 文字形态）
└─ data_contract/	← 数据契约模块（字段定义，与数据源解耦）
└─ scripts/	← 可执行脚本模块（取数引擎 / 计算引擎，自包含）
└─ config/	← 配置模块（参数 / 路由 / 版本）

六类模块的职责分别是：

- **SKILL.md**：激活指令与路由表。说明该 Skill 提供哪些模块、各自在什么条件下被激活，是外部调用方的入口索引。
- **knowledge/**：领域知识。存放公式、阈值、案例与回验基线，是本体系统里"越用越深"的主要沉淀面。
- **templates/**：输出模板。定义表格、图表、文字的输出生态，与计算结果解耦，换形态只换模板。
- **data_contract/**：数据契约。只声明"要什么字段、什么类型、什么频度"，不声明"从哪个接口取"，是数据源无关性的来源。
- **scripts/**：可执行逻辑。取数引擎、计算引擎等，要求自包含——自带依赖说明与校验脚本，可脱离 Skill 整体独立运行。
- **config/**：配置。参数、数据源路由、模块版本号，使同一套逻辑可以在不同参数配置下运行。

与 Skill 1.0 相比，关键变化不是多分了几个目录，而是**每个模块都承载独立的职责与独立的接口契约**：模块可以被单独调用、单独测试、单独版本化，不再以"整个 Skill"为唯一运行单位。

2.3 为什么不要要求 Skill 全局可用

「只取一个模块」是 Skill 2.0 的核心用法，也是放弃"Skill 必须全局可用"这一默认假设的理由。三个层面的论证：

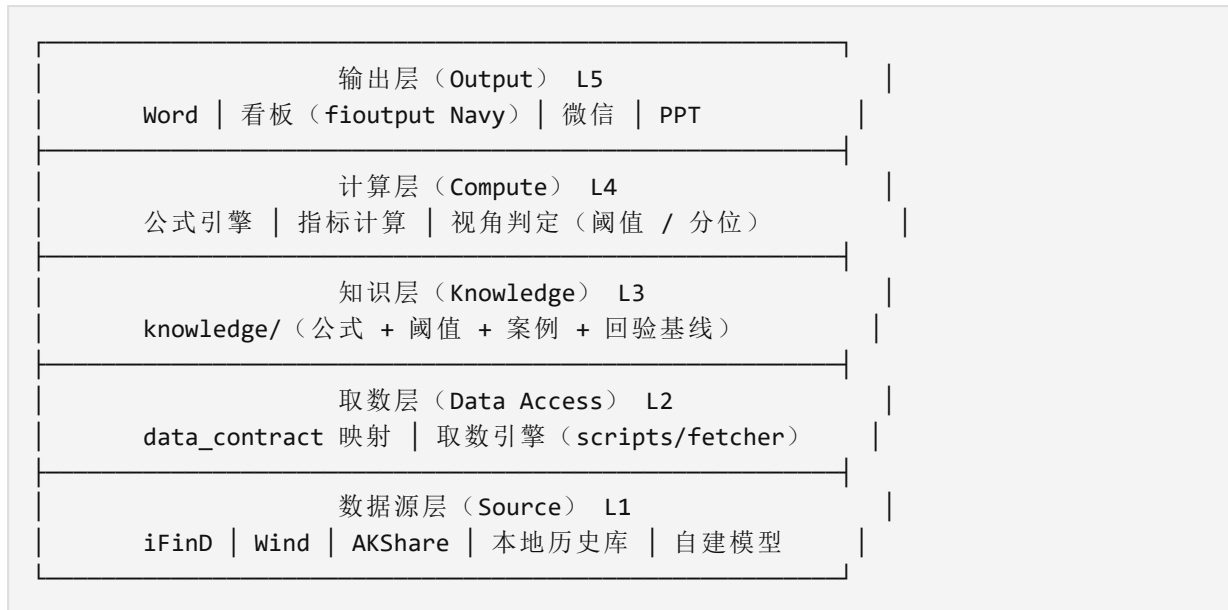
- **激活成本的权衡**：全局可用意味着每次任务都把整套能力注入上下文，而这些内容在当前任务中的命中率往往很低。模块化之后，默认不激活，调用方按路径指名加载所需模块——把"全量注入"降为"按需注入"。
- **契约独立带来独立调用**：一个模块有自己的 IN/OUT/DEP 契约，调用方只需满足其输入声明即可运行，不必理解 Skill 的其余部分。模块成为可独立交付的最小单元，其在 3.3 节给出范式。
- **跨 Skill 组合成为可能**：当复用单位从 Skill 降为模块，跨库组合的成本大幅下降。例如资金面分位这一判断逻辑，可以同时被多张研究数据库引用，而无须复制整套资金面数据库。

因此，Skill 2.0 不要求"全局可用"，而是要求"每个模块可用"。全局注册退化成为一种可选的路由策略，而不是唯一形态。

3. 中间件架构与接口契约

3.1 五层中间件

Skill 2.0 的中间件形态为五层结构，每一层独立可替换：



五层的独立性体现在两端：数据源层更换时，取数层以上的所有层不感知；输出形态变更时，计算层与知识层不感知。中间件不负责"判断"，负责把判断所需的能力以可替换、可核查的方式装配起来。

3.2 data_contract 与数据源无关性

数据源无关性由 data_contract 承担：契约只声明"我需要什么字段"，运行时由取数层完成到具体数据源的映射。以信用利差数据库为例，其 11 个分析视角对应 5 路数据源实现，共享同一份数据契约——换源不换视角。

契约通常需要声明：字段名、类型、频度、必填性，以及缺数时的容忍策略（允许缺失、补前值还是置空）。这些声明使同一契约对多个数据源成立，也使取数失败可以被契约级地诊断：问题出在缺字段、错类型，还是源停更，逐项可查。

3.3 模块 IN/OUT/DEP 契约示例

以下给出真实运行体系中的一份模块契约验收示例——资金面数据库"隔夜资金价格分位"视角：

```
# data_contract/funding_pctile.yaml — 模块：资金面·隔夜价格分位
module: funding_pctile
version: 2.1

in:                                # 输入声明：字段 + 数据源依赖
  - dr001: {type: series, freq: day, aliases: [DR001, dr001_close]}
```

```

- dr007: {type: series, freq: day, aliases: [DR007, dr007_close]}
- start: {type: date}
- window: {type: int, default: 250}

out:                                # 输出声明
- pctile: {type: scalar, unit: pct}
- regime: {type: label, enum: [宽松, 中性, 偏紧, 极紧]}

dep:                                # 运行时依赖的模块
- scripts/funding_fetcher.py        # 取数引擎（自包含）
- knowledge/资金面.md               # 阈值与回验基线
- config/funding.yaml               # 参数与路由

```

验收示例的要点在于独立调用：在 DASH 对话中针对"隔夜资金分位"发起的调用，只加载 `funding_fetcher` 这一个模块及其 `dep` 声明中的依赖，资金面数据库其余 33 个分析视角的代码不进入上下文。调用方依据契约即可运行与复核，不必通读整套数据库。

3.4 知识层回验机制：框架大于数值

知识层在公式与阈值之外，还持有**回验基线**——一组"合理区间"的历史锚点，用于在数值层面做第一道健康检查。其运行原则是"框架优先于数值"：

- 先验证框架：口径是否正确、数据链路是否完整、计算是否按契约执行；
- 再信任数值：框架无误后，若数值偏离回验基线，优先怀疑数据链路（缺数、错源、停更），而非先修改结论。

一个真实发生的例子：某周资金面分位计算值落在 90% 以上的极端区域，回验发现上游数据源缺失近三个交易日，数据补齐后该视角回归到 70% 区间。若没有回验基线，"极端值"极易被当作市场判断直接写进报告。框架正确性优先于单次数值正确性——框架一旦出错，数值再精确也不可复现、不可追溯。

4. 真实案例

4.1 模块化调用案例：DASH 只取一个模块

固定收益研究线上体系当前运行 13 个主题数据库、约 500 个分析逻辑，以及多本技术架构手册（资金面、信用利差、可转债、财务分析、基金久期）。在这一规模下，模块粒度的调用收益是直观的。

以 DASH 会话中一次资金面取数为案例：资金面数据库共 34 个分析视角，本次任务只调用其中"取数引擎"模块 `scripts/funding_fetcher.py`，其余 33 个视角不加载。由此带来的变化有三：其一，上下文占用与加载时间按比例下降；其二，调用方只依赖该模块的 `data_contract`，与其它视角的演进彻底解耦；其三，取数引擎可被其它会话独立复用，不必"借走"整套数据

库。同样的模式同样适用于信用利差数据库（11 个视角 × 5 路数据源）与可转债数据库（22 个视角）。

4.2 技术手册的拆解

Skill 1.0 时代，一本手册同时承担"给使用者看"与"给集成者看"两个角色，导致"怎么用"与"内部怎么实现"混写在一处。在 Skill 2.0 的落地中，这一步被拆开：面向使用者的介绍手册回答"做什么、怎么用"，面向集成者的架构手册回答"模块怎么组织、接口契约是什么、如何接入"。

这一拆解的意义在于，**架构手册本身就是 Skill 2.0 的一个实例**：手册中的模块图对应真实目录，接口契约对应 data_contract 与模块的 IN/OUT/DEP，集成者可以拿手册对照真实代码逐项验收。且手册中的图表并非示意画图，而是来自 DASH 对话、HTML 手册与真实跑数输出的高保真复刻，统一采用 fioutput Navy 设计体系——图表所展示的，就是生产环境实际跑出的样子。

4.3 输出形态切换

同一份计算逻辑，输出端可以是 Word 研报、HTML 看板、微信公众号图文或 PPT。这一目标在 Skill 1.0 中已有雏形（"换形态如换衣服"），但 Skill 2.0 将其落实为机制：计算层产出同一份结构化中间结果，输出层按目标形态选择 templates 模块与渲染管线——Word 走研报模板，看板走 fioutput Navy HTML 体系，微信走公众号排版，PPT 走图片与要点化组织。切换输出形态时，计算层与知识层零修改，变更集中在输出层这一个可替换面上。

5. 讨论

5.1 与 Skill 1.0 / 业界方案的对比

维度	Skill 1.0	Skill 2.0（模块化中间件）	AutoGen / LangGraph	传统终端（Bloomberg/Wind）
复用单位	整个 Skill	单个模块	Agent / 工作流	函数 / 公式
加载粒度	全局加载	按需取模块	按节点执行	按需查询
接口契约	无统一规范	IN/OUT/DEP 契约	消息 / 图边	专有 API
数据源解耦	脚本内固化	data_contract 抽象	需自行编排	终端绑定
领域知识沉淀	文档与代码混排	知识层 + 回验基线	以工具链为主	公式库

需要澄清的是，AutoGen、LangGraph 一类框架解决的是 Agent 编排与多智能体协作的问题，本身不携带投研领域知识，也不规定领域能力的封装契约。Skill 2.0 的定位在其下层——先把领域能力以模块化契约组织好，Agent 编排在此基础上叠加。两者并非替代关系。

5.2 普适性与迁移

五层中间件的抽象不依赖固收特有的业务逻辑，数据源层、取数层与输出层在跨领域时基本可直接复用，主要新增工作量在知识模块与计算模块：

目标领域	知识 / 计算模块示例	直接复用面
权益研究	行业数据库、选股因子库、财报知识库	取数层、输出层
宏观研究	经济指标库、政策事件库、预测模型库	取数层、输出层
量化策略	因子库、回测数据库、风控规则库	取数层、输出层

需要如实说明：目前仅在固收跑通全量（13 个主题数据库、约 500 个分析逻辑、多本生产运行的手册），权益、宏观、量化仍处于设计推演阶段，迁移的论据是抽象的通用性，而非完整实证。

5.3 局限性

- 模块契约的健全度依赖既有运行体系的持续维护，新建领域需要从头设计模块与契约，存在一次性成本；
- 取数引擎自包含原则带来一定程度的依赖冗余，模块间数据一致性需要中间件层显式校验；
- 知识层回验基线的覆盖深度依赖人工校验与积累，对未纳入基线的视角保护作用有限；
- 验证主场景仍为固定收益，跨领域泛化尚无完整实证支撑。

5.4 持续价值

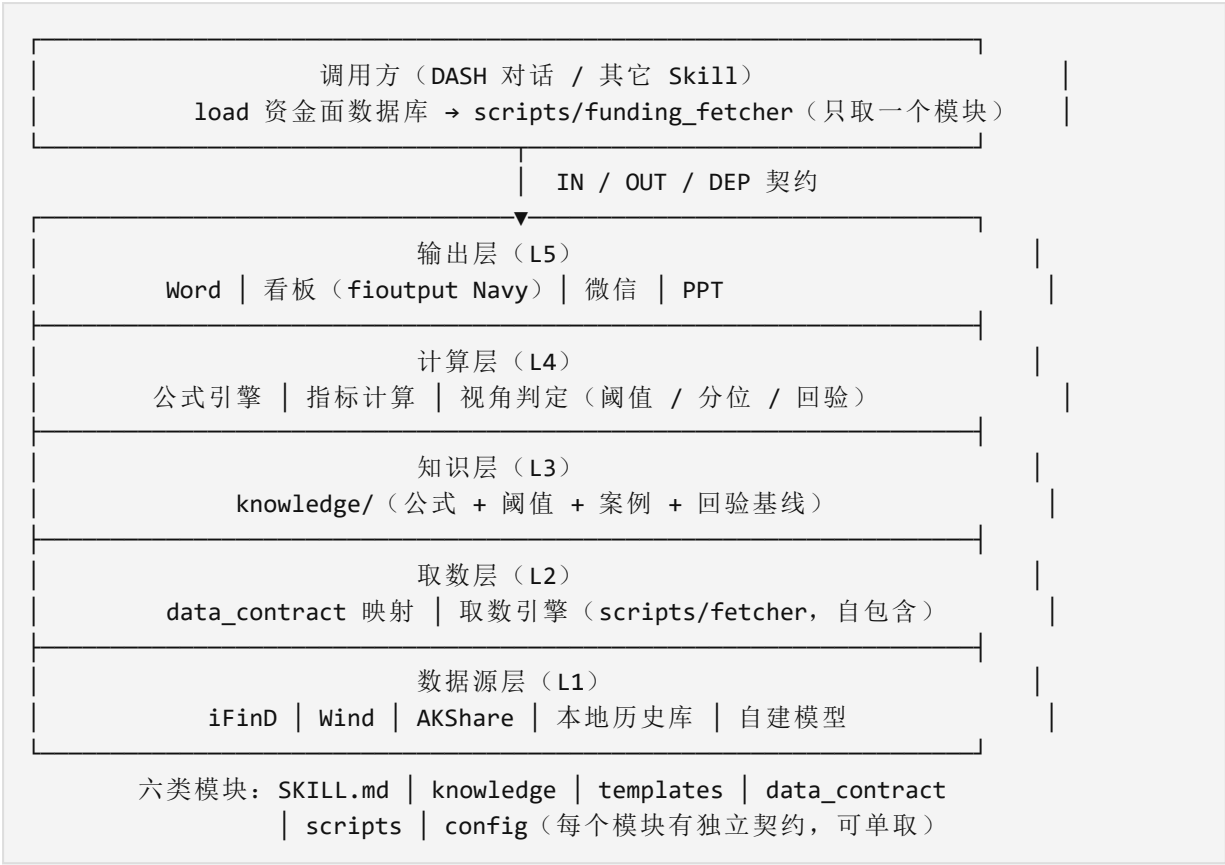
- **模块级版本管理**：单个视角的升级与回退不再牵动整个 Skill，版本漂移大幅减少；
- **跨库模块复用**：资金面分位模块被多张数据库引用，一次维护多处受益；
- **手册与代码同源**：架构手册即 Skill 2.0 实例的文档面，可随代码演进同步更新；
- **架构持续演进**：新数据源接入只改取数层，新输出形态只增输出层，中间件框架本体保持稳定。

6. 结论

从 Skill 1.0 到 Skill 2.0，核心转变是把"一个 Skill 整体可用"改为"一个模块按需可用"。模块化中间件在不改变"研究能力皆可工程化"这一立场的前提下，把封装粒度推进到模块、把契约编

码进 IN/OUT/DEP、把数据源绑定隔离进 data_contract、把数值健康检查沉淀为知识层回验基线。生产体系 13 个主题数据库、约 500 个分析逻辑的运行行为这一设计提供了实证起点；其普适性则取决于模块契约与回验机制能否在权益、宏观、量化等新领域被同样稳定地建立起来。

附录：Skill 2.0 模块化中间件架构全景图



持续更新。最新版本：github.com/timyefi/skill20-arch